

Reverse-Engineering der Transposition-Chiffre

Tamer, Yusuf
78159@studmail.htw-aalen.de

June 27, 2024

Abstract

Dieses Paper befasst sich mit der Transposition-Chiffre, bei der es um ein symmetrisches Kryptosystem handelt.

Einleitung

Moderne sowohl auch Chiffren aus der Antike basieren auf das Transposition-Verfahren. Dabei handelt es sich um eine Permutation mit der die einzelnen Zeichen vertauscht werden, sodass der eigentliche Klartext damit verborgen wird. Dieses Verfahren ist Zweidimensional, weshalb die einzelnen Zeichen auf einen 2D-Feld verteilt wird. Beispielsweise soll der folgende Klartext:

ASIMPLETRANSPPOSITION mit der Schlüssellänge 5 verschlüsselt werden.

A	S	I	M	P
L	E	T	R	A
N	S	P	O	S
I	T	I	O	N

« 4	« 3	« 2	« 1	
N	O	I	T	I
S	O	P	S	N
A	R	T	E	L
P	M	I	S	A

N	M	T	S	I
S	O	I	E	N
A	O	I	S	L
P	R	P	T	A

Programmierung der Transposition-Chiffre in C++

Formel

Mit folgenden C-Code lässt sich die Verschlüsselung sowohl auch die Entschlüsselung der Transposition-Chiffre implementieren:

```
for (int i = 0; i < plainTextSize; i++)
{
    this->cipherField[i % this->key]
    [i % (this->size/this->key)] = plainTextField[i];
}
```

Klasse

```
class transpositionCipher
{
public:
    int key;
    int size;
    char * plainField;
    char ** cipherField;
    char * decryptedField;
    transpositionCipher(int a, int b);
    void readPlainField();
    void setPlainField(char *buf);
    void encrypt();
    void decrypt();
    void readCipherField();
}
```

Das Hauptprogramm

Das Hauptprogramm lässt sich folgendermaßen implementieren:

```
int main()
{
    transpositionCipher tt(5, 20); // Übergabe vom Plaintext
    char buffer[] = { 'A', 'S', 'I', 'M', 'P', 'L', 'E', 'T', 'R', 'A', 'N', 'S', 'P', 'O', 'S', 'I', 'T', 'I', 'O', 'N' };
    tt.setPlainField(buffer);
    tt.encrypt(); // Verschlüsselungs- und Entschlüsselungsfunktion
    tt.decrypt();
    tt.printDecryptedField(); // Ausgabe vom Ciphertext
}
```

Assembler-Code vom Hauptprogramm

```

0x0000555555552800 <+0>:  endbr64
0x0000555555552804 <+4>:  push rbp
0x0000555555552808 <+8>:  mov rbp, rsp
0x000055555555280c <+12>: sub rsp, 0x40
0x0000555555552810 <+16>: mov rax, QWORD PTR fs:0x28
0x0000555555552814 <+20>: mov QWORD PTR [rbp-0x8], rax
0x0000555555552818 <+24>: xor eax, eax
0x000055555555281c <+28>: lea rax, [rbp-0x40]
0x0000555555552820 <+32>: mov edx, 0x14
0x0000555555552824 <+36>: mov esi, 0x5
0x0000555555552828 <+40>: mov rdi, rax
0x000055555555282c <+44>: call 0x555555553108 <_ZN21CC2E11>
0x0000555555552830 <+48>: movabs rax, 0x54454c504d495341

```

Vorbereitung der Parameter für den Konstruktoraufbau

Aufruf des Konstruktors

Anhand des Konstruktors wird die Größe vom CipherField und ebenso auch die KeySize festgelegt. In diesem Fall enthält das CipherField 20 Zeichen. Die KeySize beträgt 5 und damit passen in die vertikale Zeile dementsprechend nur 5 Zeichen. Daher wird der Plaintext einer Funktion übergeben, um die Verschlüsselungs-, Entschlüsselungs- und auch die Ausgabefunktion nacheinander aufzurufen.

```

0x0000555555552834 <+52>: call 0x555555553108 <_ZN21CC2E11>
0x0000555555552838 <+56>: lea rax, [rbp-0x40]
0x000055555555283c <+60>: mov rdi, rax
0x0000555555552840 <+64>: call 0x555555553108 <_ZN21CC2E11>
0x0000555555552844 <+68>: lea rax, [rbp-0x40]
0x0000555555552848 <+72>: mov rdi, rax
0x000055555555284c <+76>: call 0x555555553108 <_ZN21CC2E11>
0x0000555555552850 <+80>: call 0x555555553108 <_ZN21CC2E11>
0x0000555555552854 <+84>: lea rax, [rbp-0x40]
0x0000555555552858 <+88>: mov rdi, rax
0x000055555555285c <+92>: call 0x555555553108 <_ZN21CC2E11>
0x0000555555552860 <+96>: mov rax, 0x0
0x0000555555552864 <+100>: mov rax, QWORD PTR [rbp-0x0]
0x0000555555552868 <+104>: mov rax, QWORD PTR fs:0x28
0x000055555555286c <+108>: jmp 0x555555553108 <_ZN21CC2E11>
0x0000555555552870 <+112>: call 0x555555553108 <_ZN21CC2E11>

```

Eingabe vom Startwert

Aufruf der Verschlüsselungsfunktion

Ausgabe der Verschlüsselungsnachricht

Chronologischer Ablauf der einzelnen Funktionen, mit der die Transposition-Chiffre implementiert werden kann.

1. transpositionCipher(CipherField, KeySize)
2. setPlainField(buffer)
3. encrypt()
4. decrypt()
5. printEncryption()